

KAYNAK TÜRLERİ

- CPU
- RAM
- GPU işlem
- VRAM

İÇİNDEKİLER

Giriş & mevcut durum

GPU / VRAM

CTranslate2'ye geçiş

Tek CUDA bağlamı

OCR isteklerini birleştirme

Görüntüyü küçültme

CPU

PNG yerine JPEG

Değişmeyen kareyi atlama

oneDNN / MKLDNN

RAM

Çeviri ön belleğine sınır

Bitmap/byte[] ömrü

SQLite bakımı

Pipeline optimizasyonu

Üretici-tüketici modeli

Backpressure / kuyruk sınırı

Başlangıç ısınma isteği

Uyarlanabilir tarama aralığı

Stabilizasyon-hız dengesi

Uygulama sırası

Kaynak Kullanımını Düşürme:

CPU, RAM, GPU, VRAM

Bu doküman uygulanmadı — sadece yöntemleri ve gerekçelerini anlatıyor. Her teknik gerçek dosya yollarına ve şu anki koda referans veriyor, böylece istediğinde adım adım kendin uygulayabilirsin. Zaten yaptığımız fp16 / greedy decoding / merge-and-unload gibi değişiklikler burada tekrar anlatılmıyor; bunlar devreye girmiş durumda.

 Hedef: RTX 3060 Laptop (6 GB VRAM), tek makine, iki Python süreci

GPU & VRAM

GPU · VRAM

Şu anda iki ayrı Python süreci (çeviri sunucusu + OCR sunucusu) aynı GPU'yu paylaşıyor, ikisi de kendi CUDA bağlamını açıyor. En büyük kazanım burada.

1. OPUS-MT'yi CTranslate2 formatına dönüştür

Etki: Yüksek

Zorluk: Orta

Şu an model transformers/PyTorch üzerinden çalışıyor — genel amaçlı bir çalışma zamanı, MarianMT mimarisine özel optimize edilmemiş. **CTranslate2**, Helsinki-NLP/OPUS-MT ailesini (senin fine-tune ettiğin model dahil) native olarak destekliyor; int8 quantization ile hem VRAM hem de üretim (generate) süresini belirgin şekilde düşürüyor. Bu, PyTorch'un kendi fp16'sından bağımsız, ayrı bir kazanım katmanı.

→ tools/translation/translation_server.py

```
# 1) Modeli bir kere dönüştür (kurulum anında, çalışma zamanında değil)
# pip install ctranslate2
ct2-transformers-converter \
  --model "PsGameTranslator.App/bin/Debug/net8.0-windows10.0.19041.0/models/opus-mt"
  --output_dir "models/opus-mt-finetuned-ct2" \
  --quantization int8_float16 # GPU için: ağırlıklar int8, aktivasyonlar fp16

# 2) Sunucuda yükleme ve çeviri (transformers yerine)
import ctranslate2
import sentencepiece as spm
```

```
translator = ctranslate2.Translator("models/opus-mt-finetuned-ct2", device="cuda")
sp_source = spm.SentencePieceProcessor(model_file=".../source.spm")
sp_target = spm.SentencePieceProcessor(model_file=".../target.spm")

def translate_ct2(text: str) → str:
    tokens = sp_source.encode(text, out_type=str)
    result = translator.translate_batch([tokens], beam_size=1)
    return sp_target.decode(result[0].hypotheses[0])
```

ŞİMDİ (FP16, PYTORCH)

~460 MB model + ~700 MB sabit CUDA yükü

CTranslate2 INT8

~130-180 MB model, benzer sabit yük

Risk: int8 quantization çok nadiren 1-2 BLEU puanı kaybettirebilir — fine-tune sonrası aldığın BLEU 34.4 gibi bir referans skorun varsa dönüştürmeden sonra tekrar ölçüp karşılaştı.

2. OCR ve çeviriyi tek Python sürecinde birleştir

Etki: Yüksek

Zorluk: Orta-Yüksek

`ocr_server.py` (8765) ve `translation_server.py` (8770) iki ayrı process — her biri kendi CUDA context'ini, kendi PyTorch/Paddle runtime'ını, kendi sabit VRAM yükünü taşıyor. Aynı FastAPI uygulaması içinde iki ayrı route (`/ocr` , `/translate`) olarak çalıştırılırlarsa tek CUDA context paylaşılır, sabit yük bir kez ödenir.

→ tools/ (yeni) `unified_server.py` — `ocr_server.py` + `translation_server.py` birleşimi

```
from fastapi import FastAPI
import ocr_server, translation_server  # her ikisi de kendi router'ını export etsin

app = FastAPI()
app.include_router(ocr_server.router, prefix="")
app.include_router(translation_server.router, prefix="")

# lifespan içinde iki modeli de AYNI CUDA context'te yükle:
@app.on_event("startup")
async def startup():
    await ocr_server.load_model(device="cuda")
    await translation_server.load_model(device="cuda")
```

Not: Bu, WPF tarafında `MachineTranslationServerManager` ve `OcrServerService` 'in ikisinin de aynı port'a farklı path'lerle istek atması anlamına gelir — `appsettings.json` 'daki base URL'leri güncellemek gerekir. Orta zorlukta ama tek seferlik bir refactor.

3. Ana altyazı + konuşmacı kırpımını tek istekte birleştir

Etki: Orta

Zorluk: Düşük

Loglardan görüldüğü gibi her karede **iki ayrı HTTP OCR request** atılıyor (ana altyazı bölgesi + ikincil konuşmacı bölgesi), art arda, senkron. PaddleOCR bir seferde birden fazla görüntüyü liste olarak kabul edebiliyor — tek istekte iki görüntü gönderip GPU'ya tek seferde teslim etmek, round-trip ve Python fonksiyon çağırısı yükünü yarıya indirir.

→ `tools/ocr/ocr_server.py`

```
@app.post("/ocr/batch")
async def ocr_batch(images: list[UploadFile]):
    decoded = [decode_image(await img.read()) for img in images]
    results = engine.predict(decoded)          # tek predict() çağırısı, N görüntü
    return {"results": [format_result(r) for r in results]}
```

→ `PsGameTranslator.App/ViewModels/MonitoringViewModel.cs` — iki ayrı `RecognizeAsync` çağırısı yerine

```
var batchResult = await _ocrService.RecognizeBatchAsync(
    [frame.ImageBytes, frame.SecondarySpeakerImageBytes], "paddle", ct);
var mainResult = batchResult[0];
var speakerResult = batchResult[1];
```

4. OCR'a gönderilen görüntüyü küçült

Etki: Orta

Zorluk: Düşük

Loglarda kırpım boyutu `1920×162` gibi tam-genişlik değerler görülüyor. Altyazı metni okunaklı kalacak şekilde (yükseklik ~120-150px yeterli) yatayda %60-70'e küçültmek, hem PaddleOCR'ın işlem süresini hem VRAM'deki aktivasyon boyutunu düşürür — metin tanıma için 4K çözünürlük gereksiz.

→ `PsGameTranslator.Capture/WindowCaptureService.cs` (kırpım sonrası)

```
using var resized = new Bitmap(cropped, new Size(
    (int)(cropped.Width * 0.65),
    (int)(cropped.Height * 0.65)));
resized.Save(stream, ImageFormat.Png);
```

Risk: Çok küçük fontlu oyunlarda agresif küçültme OCR doğruluğunu düşürebilir — %65 makul bir başlangıç, test edip ayarla.

CPU

CPU

Yakalama, kırpım ve görüntü kodlama tarafında düşük riskli, hızlı kazanımlar.

5. PNG yerine JPEG kodlama

Etki: Orta

Zorluk: Düşük

PNG kayıpsız sıkıştırma CPU-yoğun; metin/altyazı bölgeleri için JPEG kalite 85-90 görsel olarak ayırt edilemez ama kodlama çok daha hızlı ve dosya küçük (HTTP transfer süresi de düşer).

→ PsGameTranslator.Capture/WindowCaptureService.cs:134 ve WgcFrameCapture.cs:296

```
var jpegEncoder = ImageCodecInfo.GetImageEncoders()
    .First(c => c.FormatID == ImageFormat.Jpeg.Guid);
var encoderParams = new EncoderParameters(1);
encoderParams.Param[0] = new EncoderParameter(Encoder.Quality, 88L);
bitmap.Save(stream, jpegEncoder, encoderParams); // ImageFormat.Png yerine
```

	PNG (ŞİMDİ)	JPEG Q88
Kodlama süresi	~15-30 ms	~4-8 ms
Dosya boyutu (1920×162 kırpım)	80-150 KB	15-30 KB

6. Değişmeyen kareyi tamamen atla

Etki: Yüksek (boşta)

Zorluk: Düşük

Loglarda zaten bir `diff_ms` alanı var — bu, kare farkını ölçüyor ama şu an sadece gecikme diagnostiği için kullanılıyor, akışı durdurmuyor. Fark eşiğin altındaysa (ekran neredeyse aynıysa — diyalog dışı sahne, duraklatılmış video, statik menü) OCR isteğini hiç atmadan bir sonraki taramaya geçmek CPU/GPU'yu boşta bırakır.

→ PsGameTranslator.App/ViewModels/MonitoringViewModel.cs (capture döngüsü)

```
var diffScore = ComputeFrameDiff(previousCrop, currentCrop);
if (diffScore < settings.MinFrameDiffThreshold)
{
    _logger.LogDebug("capture_skipped_unchanged - diff={Diff}", diffScore);
    return;    // OCR isteği hiç atılmaz, bir sonraki tarama zamanlayıcısını bekle
}
```

7. PaddleOCR için oneDNN (MKLDNN) aç

Etki: Orta (CPU modunda)

Zorluk: Düşük

Sadece OCR'ı CPU'da çalıştırmak gereken bir senaryoda (GPU çeviri modeliyle meşgulken) devreye girer — Paddle'ın Intel oneDNN backend'i CPU inference'ı belirgin hızlandırır. GPU modunda bu ayarın etkisi yok, bu yüzden az önce geri aldığımız CPU-OCR denemesiyle ilişkili.

→ tools/ocr/ocr_server.py (PaddleOCR başlatma)

```
engine = PaddleOCR(
    device="cpu",
    enable_mkldnn=True,
    cpu_threads=6,          # laptop CPU çekirdek sayına göre ayarla
)
```

RAM

RAM

Sınırsız büyüyen önbellekler ve elde tutulan görüntü buffer'ları asıl RAM artışının kaynağı.

8. Çeviri önbelleğine üst sınır ve LRU tahliye

Etki: Orta (uzun oturumlarda yüksek)

Zorluk: Düşük

`translation_cache.json` şu an 3700+ kayıt taşıyor ve sınırsız büyüyor — her uzun oynanış oturumu dosyayı ve bellekteki sözlüğü büyütmeye devam ediyor. Sabit boyutlu bir LRU (en az kullanılanı at) yapısı bunu sınırlar.

→ PsGameTranslator.Infrastructure/Translation/TranslationCache.cs

```
private const int MaxEntries = 8000;

public void Set(string key, string value)
{
    if (_cache.Count ≥ MaxEntries && !_cache.ContainsKey(key))
    {
        var oldest = _accessOrder.First(); // LinkedList ile LRU sırası tutulur
        _cache.Remove(oldest);
        _accessOrder.RemoveFirst();
    }
    _cache[key] = value;
    TouchAccessOrder(key);
}
```

9. Bitmap ve byte[] buffer ömrünü kısalt

Etki: Orta

Zorluk: Düşük

Her karede birden fazla `Bitmap` / `byte[]` üretiliyor (tam kare, ana kırpım, ikincil kırpım). `using` bloklarıyla erken serbest bırakmak ve büyük array'leri `ArrayPool<byte>` ile kiralamak GC baskısını ve peak RAM'i düşürür.

→ `PsGameTranslator.Capture/WgcFrameCapture.cs`

```
var buffer = ArrayPool<byte>.Shared.Rent(estimatedSize);
try
{
    var written = EncodeToBuffer(bitmap, buffer);
    return buffer.AsSpan(0, written).ToArray(); // sadece gerçek boyut kopyalanır
}
finally
{
    ArrayPool<byte>.Shared.Return(buffer);
}
```

10. SQLite periyodik VACUUM

Etki: Düşük-Orta

Zorluk: Düşük

`translation_memory.db` silme/güncelleme işlemleriyle parçalanır (fragmentation), disk ve sayfa önbelleği RAM kullanımı gereğinden büyük kalır. Uygulama kapanışında bir kez `VACUUM` çalıştırmak dosyayı sıkıştırır.

→ `PsGameTranslator.Infrastructure/Translation/SqliteTranslationMemoryService.cs` (Dispose)

```
public void Dispose()
{
    using var cmd = _connection.CreateCommand();
    cmd.CommandText = "VACUUM;";
    cmd.ExecuteNonQuery();
    _connection.Dispose();
}
```

Pipeline optimizasyonu GENEL

Tek bir aşamayı hızlandırmak yerine aşamalar arasındaki bekleme ve gereksiz işi ortadan kaldırmak — genelde tek bir optimizasyondan daha büyük toplam kazanç verir, çünkü darboğaz tek bir yerde değil, aşamaların birbirini *senkron* beklemesinde.

11. Aşamaları üretici-tüketici (producer-consumer) modeline taşı

Etki: Yüksek

Zorluk: Orta-Yüksek

Şu anki akış büyük ölçüde **sıralı**: yakala → kırp → OCR bekle → sınıflandır → çeviri kuyruğuna koy. Bir sonraki karenin yakalanması, mevcut karenin OCR'ı bitene kadar büyük ölçüde bekliyor (loglarda `capture_to_ocr_done_ms` 'un tek bir OCR isteğinden çok daha yüksek çıkması bunun izi). Yakalama, OCR ve çeviriyi ayrı `Channel<T>` kuyruklarıyla birbirinden ayrık task'lara bölmek, bir aşama meşgulken diğerinin boş beklemesini engeller — GPU OCR ile meşgulken CPU zaten bir sonraki kareyi yakalayıp kırpılmış olur.

→ PsGameTranslator.App/ViewModels/MonitoringViewModel.cs (yeniden yapılanma)

```
private readonly Channel<CapturedFrame> _captureChannel =
    Channel.CreateBounded<CapturedFrame>(new BoundedChannelOptions(2)
    {
        FullMode = BoundedChannelFullMode.DropOldest, // bkz. #12 backpressure
    });

// Görev 1: sadece yakala + kırp, sonucu kanala yaz
async Task CaptureLoopAsync(Cancellation_token ct)
{
    while (!ct.IsCancellationRequested)
    {
        var frame = await CaptureAndCropAsync(ct);
```

```

        await _captureChannel.Writer.WriteAsync(frame, ct);
        await Task.Delay(CurrentIntervalMs, ct);
    }
}

// Görev 2: kanaldan oku, OCR + sınıflandırma + çeviri kuyruğu – paralel çalışır
async Task ProcessLoopAsync(CancellationToken ct)
{
    await foreach (var frame in _captureChannel.Reader.ReadAllAsync(ct))
        await RunOcrAndDispatchAsync(frame, ct);
}

// Başlangıçta ikisini de bağımsız başlat:
_ = Task.Run(() => CaptureLoopAsync(ct));
_ = Task.Run(() => ProcessLoopAsync(ct));

```

12. Backpressure: kuyruk sınırı ve en yeni kareyi tutma

Etki: Yüksek (aşırı yük altında)

Zorluk: Düşük

OCR'ı CPU'ya aldığımız denemede tam olarak gördüğümüz şey buydu: OCR süresi kare aralığından uzun sürünce istekler kuyrukta birikti, gecikme 561ms'den 4900ms'ye kadar tırmandı. Sınırsız bir kuyruk, geç kalan sistemi *daha da* geciktirir çünkü eski, artık alakasız kareleri bile işlemeye devam eder. Sınırlı kapasiteli bir kanal + **DropOldest** politikası, sistem yetişemediğinde en eski (zaten bayatlamış) kareyi atıp en güncel kareye odaklanır.

→ Yukarıdaki `BoundedChannelOptions(2)` — kapasite 2, dolunca en eskisini at

```

// Sınırsız Queue<T> yerine:
var options = new BoundedChannelOptions(capacity: 2)
{
    FullMode = BoundedChannelFullMode.DropOldest,
    SingleReader = true,
    SingleWriter = true,
};

```

Bu yaşadığımız gerçek bir olaydı: OCR'ı CPU'ya taşıdığımızda arka arkaya biriken isteklerin gecikmesi katlanarak arttı — backpressure olsaydı sistem "yetişemiyorum" deyip en son kareye atlardı, birikme yerine.

13. Başlangıçta ısınma (warm-up) isteği at

Etki: Orta (ilk istek gecikmesi)

Zorluk: Düşük

CUDA'nın ilk çekirdek derlemesi/yükü ("kernel warm-up") ilk isteği belirgin şekilde yavaşlatır — bu oturumda ölçüm yaparken de her seferinde önce görünmez bir "warmup" isteği attık, tam da bu yüzden. Sunucu açılışında gerçek bir çeviri/OCR isteğini sahte bir metinle bir kez tetiklemek, kullanıcının göreceği *ilk gerçek* altyazının o soğuk-başlangıç gecikmesini yememesini sağlar.

→ tools/translation/translation_server.py (lifespan, model yüklendikten hemen sonra)

```
_model.eval()
elapsed = (time.perf_counter() - t0) * 1000
print(f"[translation_server] Model loaded in {elapsed:.0f} ms", flush=True)

# Isınma: ilk gerçek isteğin CUDA kernel derleme maliyetini ödememesi için
with torch.no_grad():
    _ = _model.generate(**_tokenizer("warmup", return_tensors="pt").to(_device),
                        max_new_tokens=4, num_beams=1)
print("[translation_server] Warm-up complete.", flush=True)
```

14. Sahne durgunken tarama aralığını uzat

Etki: Yüksek (boşta senaryoda)

Zorluk: Orta

Şu an tarama aralığı sabit (~300-400ms), diyalog olsun olmasın. Art arda birkaç kare "değişiklik yok / metin yok" dönerse aralığı kademeli arttırıp (örn. 400ms → 800ms → 1500ms), yeni bir değişiklik algılanınca hemen 400ms'e dönmek — boşta CPU/GPU tüketimini büyük ölçüde azaltır, diyalog başladığında tepki hızını korur.

→ PsGameTranslator.App/ViewModels/MonitoringViewModel.cs (tarama zamanlayıcısı)

```
private int _consecutiveEmptyFrames;
private int CurrentIntervalMs =>
    _consecutiveEmptyFrames switch
    {
        ≤ 2 => 400,
        ≤ 6 => 800,
        _    => 1500,
    };

// her boş OCR sonucunda: _consecutiveEmptyFrames++;
// her dolu sonuçta:      _consecutiveEmptyFrames = 0;
```

15. Stabilizasyon süresini içerik uzunluğuna göre uyarlama

Etki: Orta

Zorluk: Orta

Şu anki 250ms sabit — kısa repliklerde ("Hayır.") gereksiz beklemeye, uzun diyalog daktilo efektinde bazen yetersiz kalmaya sebep olabilir. Metin uzunluğuna göre orantılı bir bekleme (kısa metne az, uzun metne biraz daha fazla) hem tepki hızını hem doğruluğu iyileştirebilir.

→ PsGameTranslator.Infrastructure/Translation/OrderedSubtitlePipeline.cs (HandleStabilization)

```
var stabilizationMs = Math.Clamp(  
    150 + text.Length * 2,    // kısa metin ~150-200ms, uzun metin ~350-400ms  
    150, 400);
```

Bu ayar önceki oturumda bilerek sabit 250ms'de bıraktık (kullanıcı isteğiyle) — burada sadece bir sonraki iyileştirme fikri olarak listeleniyor, otomatik uygulanmadı.

Uygulama sırası

Etki/zorluk oranına göre gerçek bir öncelik sırası — yukarıdan aşağı uygulamaları önerilir.

1

Backpressure / kuyruk sınırı ekle (#12)

Riski yok, sadece iyileştirir — OCR-CPU denemesinde yaşadığımız birikme sorununu tekrar yaşamamamızı garantiler.

2

OCR isteklerini birleştir (#3)

Tek kod değişikliği, hemen ölçülebilir round-trip kazancı.

3

Görüntüyü küçült + JPEG'e geç (#4, #5)

İkisi birlikte kırpım→OCR yolunun CPU maliyetini büyük ölçüde düşürür.

4

Başlangıç ısınma isteği (#13)

Beş satırlık kod, ilk çevirinin soğuk-başlangıç gecikmesini yok eder.

5

Uyarlanabilir tarama aralığı (#14)

Boşta senaryoda (menü, ara sahne) en büyük genel kazanım.

6

Çeviri ön belleğine LRU sınırı (#8)

Uzun oturumlarda RAM artışını durdurur, riski yok.

7

Üretici-tüketici pipeline'a geçiş (#11)

En büyük mimari kazanım ama en fazla test gerektiren refactor.

8

CTranslate2'ye geçiş (#1)

En yüksek etki ama BLEU'yu tekrar ölçmeyi gerektirir — ayrı bir oturumda ele alınmalı.

9

OCR + çeviriyi tek sürece birleştir (#2)

En büyük VRAM kazancı ama en riskli refactor — en sona bırakılmalı.

Bu rehber PsGameTranslator kod tabanının 2026-07-13 tarihli hâline göre yazıldı — dosya yolları ve satır referansları o ana kadar yapılan değişiklikleri (fp16, greedy decoding, model merge, OCR okuma sırası düzeltmesi) yansıtır. Hiçbir kod burada otomatik uygulanmadı.